

Sprint 3 Plan

SlugFound

CSE 115

Sprint Completion Date:	TBD
Revision:	1.0
Revision Date:	May 10, 2025

1. Sprint Goal

Complete the SlugFound platform by delivering four high-impact features: fully functional real-time messaging between users, geolocation tagging on all item posts, AI-powered listing creation from a single photo using the Google Gemini Vision API, and AI-powered photo-match search that compares an uploaded image against the database to surface likely matches. By the end of this sprint, SlugFound should feel like a complete, intelligent lost-and-found platform for the UCSC community.

2. Task Listing (by User Story)

Total estimated hours this sprint: 46 hours

User Story 3.1 [High] [5 pts]

"As a logged-in user, I want to send and receive real-time messages with other students about a lost or found item so that we can coordinate returning it quickly."

Specifications & Requirements

- Messages table: Create a public.messages table with columns: id (uuid, PK), conversation_id (uuid, FK), sender_id (FK to profiles.id), recipient_id (FK to profiles.id), item_id (FK to items.id, nullable), content (text, max 2000 chars), is_read (boolean, default false), created_at (timestampz).
- Conversations table: Create a public.conversations table with columns: id (uuid, PK), item_id (FK to items.id), participant_1 (FK to profiles.id), participant_2 (FK to profiles.id), last_message_at (timestampz). Unique constraint on (participant_1, participant_2, item_id).
- Real-time delivery: Use Supabase Realtime postgres_changes subscription on the messages table. New messages appear instantly without page refresh.
- Conversation initiation: Users can start a conversation from any item card via a 'Message Poster' button. If a conversation already exists for that item + user pair, navigate to the existing thread.
- Unread indicator: Show an unread badge count on the Messages nav item. Marking a conversation as read sets is_read = true for all messages where recipient_id = current user.
- RLS policies: Users can only SELECT messages where they are sender or recipient. Users can only INSERT messages where sender_id = their own auth ID.
- UI: Left panel shows conversation list with avatar, display name, item title, last message preview, and timestamp. Right panel shows full message thread with send input.

Tasks

- Design and write CREATE TABLE migrations for messages and conversations tables (2 hrs)
- Write RLS policies for both tables and verify in Supabase SQL editor (1 hr)
- Wire conversation list panel to real Supabase data, ordered by last_message_at (2 hrs)

- Implement send message functionality with Supabase insert and optimistic UI update (2 hrs)
- Subscribe to Supabase Realtime on messages table for live delivery (2 hrs)
- Add unread badge counter on sidebar nav icon and mark-as-read logic (1 hr)

Total for User Story 3.1: 10 hours

Definition of Done

- A user can open an item listing and tap 'Message Poster' to start or resume a conversation
- Messages are delivered in real-time to both participants without a page refresh
- Conversation list displays the correct last message preview and timestamp
- Unread badge count on the sidebar updates when new messages arrive and clears when the thread is viewed
- RLS prevents users from reading or sending messages they are not a party to (verified in Supabase SQL editor)
- All migration files committed to /supabase/migrations/ and all code merged to main with a passing build

User Story 3.2 [High] [5 pts]

"As a user posting a lost or found item, I want to tag the exact location on a map so that others can understand precisely where the item was found or last seen."

Specifications & Requirements

- Database: Add latitude (float8, nullable) and longitude (float8, nullable) columns to the public.items table via a new Supabase SQL migration.
- Map library: Use react-leaflet with OpenStreetMap tiles (no API key required). Do not use Google Maps (requires billing setup).
- Post form — auto-detect: Add a 'Use My Location' button on the post form that calls the browser Geolocation API (navigator.geolocation.getCurrentPosition). Show a loading spinner while fetching. Display a human-readable address via OpenStreetMap Nominatim reverse geocoding (free, no key).
- Post form — manual pin drop: Embed a react-leaflet map below the location field. User can drag a pin to set coordinates manually. Coordinates update in real-time as the pin is dragged.
- Storage: On form submit, include latitude and longitude in the Supabase items.insert() call.
- Item card: Show a small location pill with the reverse-geocoded neighborhood/building name if coordinates exist.
- Fallback: If user denies geolocation permission, keep the text location field from Sprint 2 as the fallback. Geotagging is optional, not required.

Tasks

- Write and apply Supabase migration to add lat/long columns to items table (1 hr)
- Integrate browser Geolocation API on post form with loading and permission-denied states (2 hrs)
- Embed react-leaflet draggable pin map on post form and wire to form state (3 hrs)
- Add Nominatim reverse geocoding to display human-readable location name (1 hr)
- Update items.insert() to persist coordinates and update item card to display location pill (1 hr)

Total for User Story 3.2: 8 hours

Definition of Done

- User can tap 'Use My Location' and have coordinates auto-populated on the post form

- User can manually drag a map pin to set coordinates if geolocation is denied or inaccurate
- Submitted posts include lat and lng values stored in the items table
- Item cards display a human-readable location label derived from coordinates where available
- If the user skips geotagging entirely, the form still submits successfully with null lat/lng
- All migration files are committed and code is merged to main with a passing build

User Story 3.3 [High] [8 pts]

"As a user who found an item, I want to upload a photo and have the listing form automatically filled in by AI so that I can report the item quickly without manually describing it."

Specifications & Requirements

- AI provider: Google Gemini API using the gemini-2.0-flash model via the @google/generative-ai JavaScript SDK. API key stored in a server-side Next.js environment variable (GEMINI_API_KEY) — never exposed to the client.
- API route: Create a Next.js API route at /api/ai/scan-item. Accepts a base64-encoded image. Sends image to Gemini with a structured prompt requesting a JSON response with fields: title (string, max 80 chars), category (one of the predefined Sprint 2 categories), description (string, max 500 chars), suggested_type ('found' | 'lost').
- Prompt engineering: The prompt must instruct Gemini to respond ONLY with a valid JSON object and nothing else. Include field constraints and the exact allowed category values in the prompt.
- UI — Scan with AI button: Add a 'Scan with AI' button to the post form adjacent to the photo upload drop zone. Button is only active after a photo has been selected.
- Loading state: While Gemini is processing, disable the form and show an animated AI scanning indicator with the message 'Analyzing your photo...'.
- Form injection: On success, populate the title, category, and description fields with Gemini's response. Highlight auto-filled fields with a light blue border so the user knows which fields were AI-generated. User can edit any field before submitting.
- Fallback: If Gemini returns an unparseable response or an error, show a toast notification ('AI scan failed — please fill in the fields manually') and leave the form unchanged.
- Privacy: The uploaded image is only sent to Gemini for analysis. It is not stored or transmitted anywhere else until the user submits the full form.

Tasks

- Install @google/generative-ai SDK and configure GEMINI_API_KEY environment variable (1 hr)
- Create Next.js API route /api/ai/scan-item with base64 image ingestion and Gemini call (3 hrs)
- Write and iterate on the structured JSON prompt for item field extraction (2 hrs)
- Build 'Scan with AI' button on post form with active/disabled state tied to photo upload (1 hr)
- Implement form field injection with AI-highlight styling and user-editable override (2 hrs)
- Add loading state, error toast, and fallback handling (1 hr)

Total for User Story 3.3: 10 hours

Definition of Done

- User uploads a photo and clicks 'Scan with AI'; the title, category, and description fields are auto-populated within 5 seconds
- Auto-filled fields are visually distinguished from manually entered fields
- User can edit any AI-populated field before submitting
- The GEMINI_API_KEY is never exposed in client-side JavaScript or network requests from the browser
- If Gemini fails or returns invalid JSON, the form remains usable and a clear error toast is shown

- Gemini API route returns a 400 if no image is provided and a 500 with a safe error message if Gemini is unavailable
- All code merged to main on GitHub with a passing build

User Story 3.4 [High] [8 pts]

"As a student who has lost an item, I want to upload a photo of it and have the app automatically search for matching found items in the database so that I can quickly see if someone has already turned it in."

Specifications & Requirements

- AI provider: Google Gemini API (gemini-2.0-flash) via a server-side Next.js API route at `/api/ai/match-item`. Accepts a base64-encoded image and returns a structured JSON descriptor object.
- Descriptor extraction: Gemini analyzes the photo and returns a JSON object with: category (one of the predefined categories), keywords (array of up to 8 descriptive strings, e.g. ['blue', 'backpack', 'zipper', 'UCSC']), color (primary color string), and size_estimate ('small' | 'medium' | 'large').
- Database matching: Use the extracted category and keywords to query the items table. Filter by `type='found'` and category match first. Then use Supabase `.ilike()` to search title and description against each extracted keyword. Return results ordered by number of keyword hits (descending), then by `created_at` descending.
- Results display: Show up to 10 matching items in a ranked list with a match strength badge (High / Medium / Low) based on number of keyword hits (5+ = High, 2–4 = Medium, 1 = Low). Display item photo, title, location, date, and poster name.
- UI entry point: Add a 'Find by Photo' button on both the Lost Items page and the landing page (for unauthenticated users to see the feature). The search modal is accessible to authenticated users only; unauthenticated users are prompted to sign in.
- No-match state: If no items meet even 1 keyword match, show an empty state with the message 'No matching items found — try posting a lost item report so finders can contact you.'
- Privacy: Same as US 3.3 — uploaded image is only sent to Gemini and is not stored.

Tasks

- Create Next.js API route `/api/ai/match-item` with Gemini descriptor extraction and JSON parsing (3 hrs)
- Write and iterate on the Gemini descriptor prompt for reliable structured output (2 hrs)
- Build keyword-based Supabase query logic with keyword-hit scoring and ordering (3 hrs)
- Build 'Find by Photo' modal UI with photo upload, loading state, and results list (2 hrs)
- Implement match strength badge logic and result card layout (1 hr)
- Add no-match empty state and error handling (1 hr)

Total for User Story 3.4: 12 hours

Definition of Done

- User uploads a photo and receives a ranked list of potentially matching found items within 8 seconds
- Results are ordered by match strength and each card displays a High / Medium / Low badge
- Searching with a photo of a common item (e.g. a blue water bottle) returns only items with `type='found'` from the database
- Unauthenticated users who click 'Find by Photo' are redirected to sign in before accessing the feature
- If no keyword matches exist in the database, the empty state message is displayed

- The GEMINI_API_KEY is never exposed to the client; all Gemini calls go through the server-side API route
- All code merged to main on GitHub with a passing build

User Story 3.5 [Medium] [3 pts]

"As a user browsing lost or found items, I want to switch to a map view so that I can visually see where items were found or lost on the UCSC campus."

Specifications & Requirements

- Map toggle: Add a Map / List toggle button group to both the Found Items and Lost Items pages, consistent with the existing search/filter bar design.
- Map library: Use the same react-leaflet + OpenStreetMap setup from US 3.2. Initialize the map centered on UCSC's coordinates (36.9916° N, 122.0583° W) at zoom level 15.
- Item pins: Render a map pin for each item that has lat/lng data. Items without coordinates are excluded from map view. Show the total count of items with coordinates vs. total items in an info bar above the map.
- Pin popup: Clicking a pin opens a popup with the item photo thumbnail, title, category badge, date, and a 'View Item' link that opens the item detail page.
- Filters apply to map: Applying the search bar or category filter updates the map pins in real-time, the same as it updates the list view.
- Responsive: Map view fills available width and is at least 500px tall on desktop and full-screen on mobile.

Tasks

- Add Map / List toggle control to Found and Lost pages (1 hr)
- Render react-leaflet map with UCSC center and item pins from Supabase query results (2 hrs)
- Build pin popup component with item photo, title, category, and View Item link (1 hr)
- Ensure search and category filter updates apply to map pins in real-time (1 hr)
- Add info bar showing count of geotagged items vs. total items (1 hr)

Total for User Story 3.5: 6 hours

Definition of Done

- Found Items and Lost Items pages each have a working Map / List toggle
- Map renders centered on UCSC campus with pins for all geotagged items of the correct type
- Clicking any pin opens a popup with correct item details and a working View Item link
- Applying search or category filters removes non-matching pins from the map in real-time
- Items without lat/lng coordinates are gracefully excluded from map view without errors
- All code merged to main on GitHub with a passing build

3. Team Roles

Team Member	Email	Role(s)
Bardia Nasab	bnasab@ucsc.edu	Scrum Master
Colin Posat	cposat@ucsc.edu	Developer, UI/UX Lead
Sam Madinya	smadinya@ucsc.edu	Developer

Aidan Nguyen	anguy480@ucsc.edu	Developer
--------------	-------------------	-----------

4. Initial Task Assignment

Team Member	User Story	Initial Task
Bardia Nasab	US 3.1	Design messages table schema and write CREATE TABLE migration
Sam Madinya	US 3.2	Add lat/ing columns to items table and integrate browser Geolocation API on post form
Aidan Nguyen	US 3.3	Set up Gemini API client and build AI scan endpoint for photo-to-form fill
Colin Posat	US 3.4	Build Find by Photo UI and Gemini image descriptor extraction flow

5. Initial Scrum Board

The Sprint 3 scrum board is managed in Jira

6. Initial Burnup Chart

The Sprint 3 burnup chart is tracked in Jira

Total sprint capacity: 29 story points across 46 estimated hours.

7. Scrum Meeting Times

Meeting #	Day	Time	Location	TA/Tutor Visit?
1	Mondays	12:00 – 1:00 PM	Zoom	Yes — Lab session
2	Wednesdays	5:30 – 6:30 PM	TBD	No
3	Fridays	12:00 – 1:00 PM	TBD	No